■■ AgriTrace Technology Explanation Manual

A Plain-English Guide to Blockchain Ledger, Multi-Agent LangGraph AI, YOLOv8 Crop Detection, and PyTorch Vision Transformers

Target Audience: Business stakeholders, non-technical developers, and system auditors.
Focus: Simplification of core technical components without losing engineering context.

1. Polygon Amoy Blockchain Ledger

What is it in simple words?

Think of the blockchain as a *shared public registry book*. Every farmer, transport coordinator, and checker writes transactions down in it. Once a page in this book is filled out, signed, and stamped, it is sealed. No single person can erase, change, or rip out that page because copies of the same book are held by thousands of independent computers worldwide. It provides absolute trust and proof of authenticity.

How it works in AgriTrace:

When a farmer registers a crop harvest batch on our portal, the system compiles the harvest details (Batch ID, Farmer ID, Crop Name, Harvest Date, and Land Location) into a text string and computes its unique digital fingerprint (a SHA-256 hash). The backend then signs a transaction with a private key and uploads this fingerprint on-chain. When a buyer scans the QR code on the crop card, the backend queries the Polygon blockchain to fetch the saved fingerprint. If anyone tried to modify the crop details in our database, the newly generated fingerprint will mismatch the blockchain copy, triggering a **TAMPERED** security warning on the portal.

2. Privacy-Aware LangGraph Multi-Agent Chatbot

What is it in simple words?

Imagine you have a group of specialized agricultural consultants sitting in a room (e.g. a weather expert, a market price expert, a crop doctor, a database manager). Instead of chatting with them one-by-one, you talk to the **Supervisor** (the manager). The supervisor listens to your query, decides who needs to answer next, coordinates the workflow, and combines all their notes into one intelligent advisory report. This dynamic workflow is orchestrated using *LangGraph*.

Privacy Awareness Integration:

In a public supply chain, we must protect the farmer's personal safety and property. If a buyer queries the chatbot about a crop batch location, the system is designed to be *privacy-aware*. The RAG and weather agents fuzzed or generalize spatial coordinates: instead of returning the exact latitude/longitude or home coordinates of the farmer, the chatbot filters the context to only output the village name or district, hiding exact GPS bounds from unauthorized roles.

3. YOLOv8 & PyTorch Vision Transformers (ViT)

What is it in simple words?

These are the *eyes* of AgriTrace. They allow the server to process, recognize, and diagnose crop photos.

How it works:

YOLOv8 (You Only Look Once v8): This is an object detection model. When a farmer uploads a leaf photo, YOLOv8 instantly detects where the leaf or crop is in the frame, drawing a bounding box around it. It isolates the leaf from background noise (like hands or soil).

ViT (Vision Transformer): While YOLOv8 locates the leaf, ViT diagnoses it. Built on PyTorch, the Vision Transformer model behaves like natural language transformers but applies attention heads directly to grid patches of the leaf image. It detects tiny leaf lesions, yellowing, and spotting patterns, outputting the exact disease classification (e.g. Early Blight vs Late Blight), severity indexes, and treatment remedies.

Multimodal Validation: To prevent farmers from registering a 'Carrot' crop but uploading a photo of a tomato leaf or random objects, the backend utilizes a multimodal visual gatekeeper. It cross-checks the visual content of the image with the entered crop name, rejecting fraudulent entries before they are logged on the blockchain.

4. Solidity & Web3.py

What is it in simple words?

If the blockchain is a secured registry, *Solidity* is the language used to write the registry's rules, and *Web3.py* is the phone line our backend server uses to talk to it.

How they interact:

Solidity Smart Contract: We write the crop registry logic in a smart contract. It establishes how fingerprints are stored on-chain. Once deployed onto the Polygon Amoy network, this contract runs autonomously and cannot be changed.

Web3.py: The FastAPI backend is written in Python, but the blockchain nodes speak Ethereum JSON-RPC. Web3.py acts as the connection bridge. It formats the backend transaction requests, uses private keys to sign transaction payloads securely, sends them to the Polygon blockchain RPC gateway, and reads the on-chain ledger variables whenever a QR code verification request is received.